

Plataforma de Recomendação com Neo4j, Redis e Python

Danillo Vidal - 572039

Guilherme Buosi - 571661

Guilherme Rigobello - 569376

Mateus Henrique - 569331

Pedro Samugim - 568920

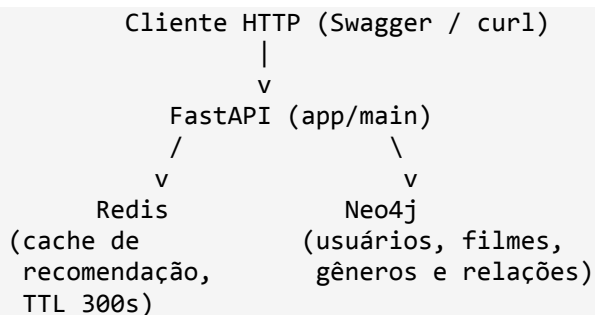
O que o projeto faz

- Cadastra usuários, filmes e gêneros
- Registra filmes assistidos e avaliações
- Gera recomendações personalizadas com base nos gêneros que o usuário mais avaliou bem
- Armazena as recomendações em cache no Redis (com TTL)
- Identifica CACHE HIT e CACHE MISS
- Permite invalidar o cache

Tecnologias

- Neo4j Aura (banco de grafos, free tier)
- Redis (rodando via Docker)
- FastAPI (Python)
- Docker Compose (para subir o Redis local)

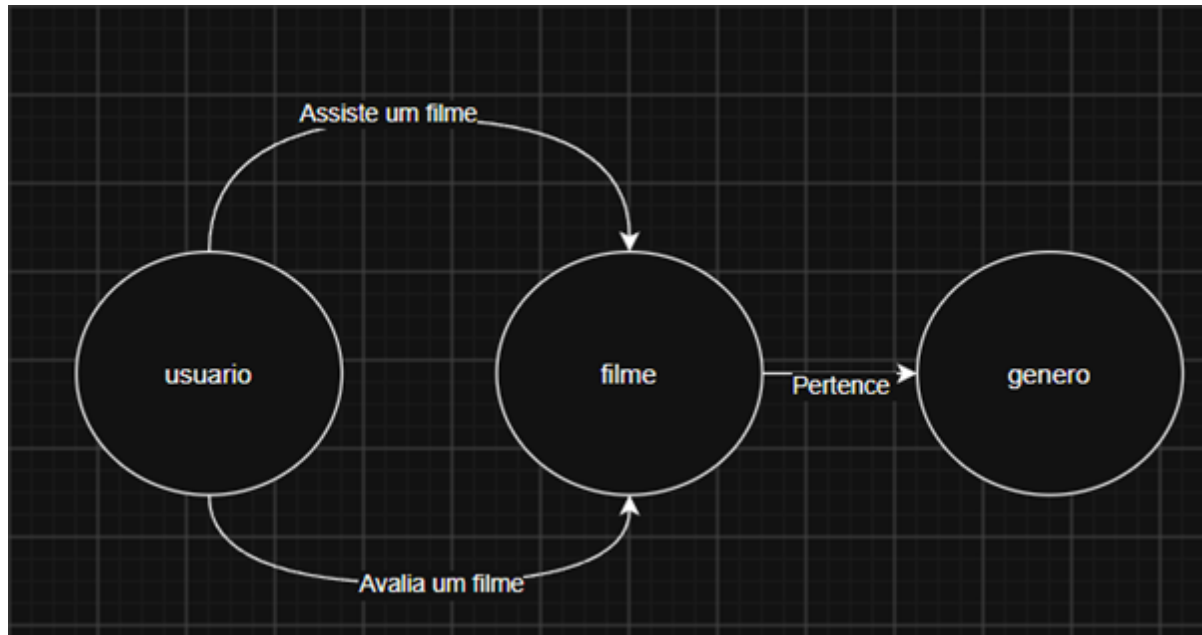
Arquitetura



Fluxo de GET `/recomendacoes/{usuario_id}`:

1. Monta a chave `recomendacao:{usuario_id}` e verifica se existe no Redis.
2. Se existir → **CACHE HIT**: retorna direto, sem consultar o Neo4j.
3. Se não existir → **CACHE MISS**: consulta o Neo4j, calcula as recomendações, salva no Redis com TTL de 300s e retorna.
4. DELETE `/recomendacoes/{usuario_id}/cache` invalida o cache manualmente, forçando o próximo pedido a ser MISS.

Modelagem do grafo



Interface de um banco de dados de grafos (Cypher) mostrando a execução de uma consulta e o resultado visualizado.

Database information

- Nodes (23): Filme, Genero, Usuario
- Relationships (30): ASSISTIU, AVALIOU, PERTENCE_A
- Property keys: ano, data, data_nascimento, email, id, idade, nome, nota, titulo

Query: `d63674a5$ MATCH p=()-[:ASSISTIU]->() RETURN p LIMIT 25;`

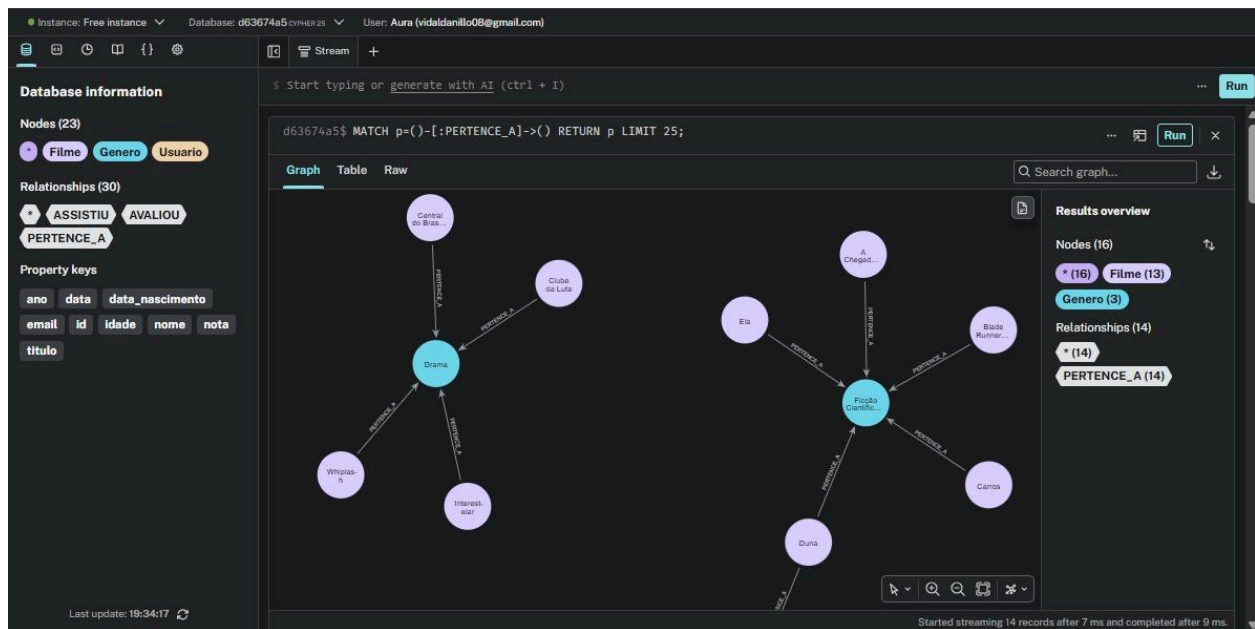
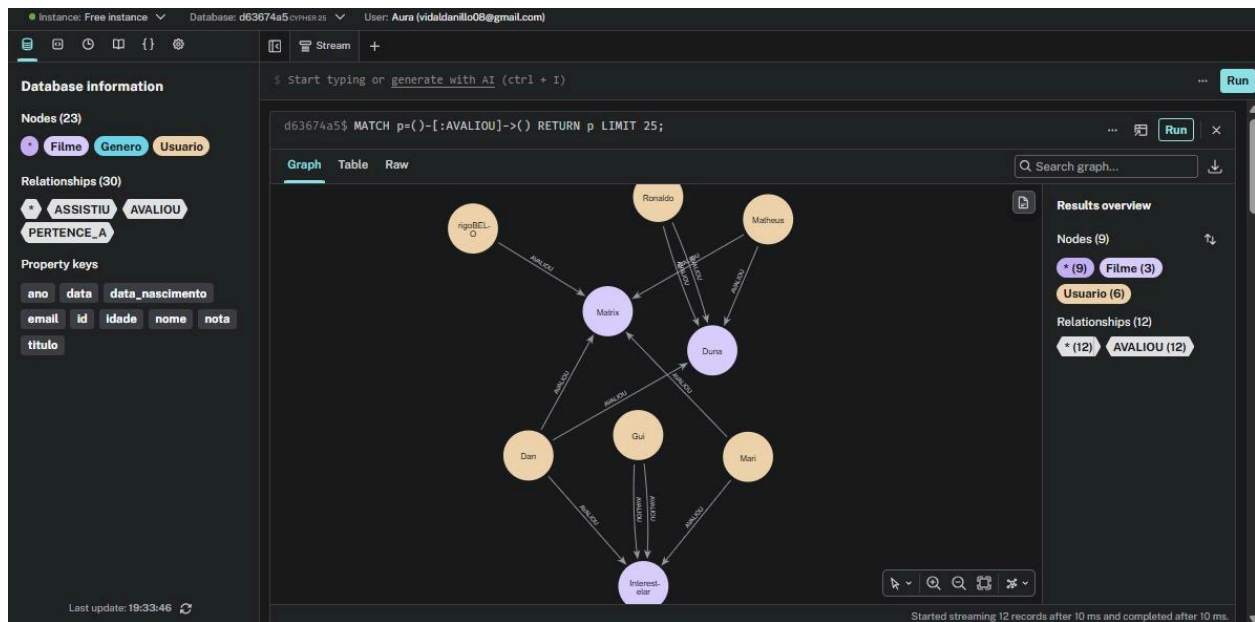
Results overview:

- Nodes (6): Filme (2), Usuario (4)
- Relationships (4): ASSISTIU (4)

Visualização do grafo: O grafo mostra os seguintes nós e relações:

- Nós: Duna (filme), Intercontinental (filme), 2007-01-15 (filme), 2003-08-22 (filme), 2008-03-07 (filme), 2003-05-10 (filme).
- Relações: Duna (filme) está relacionado a 2007-01-15 (filme) via `ASSISTIU`; Duna (filme) está relacionado a 2008-03-07 (filme) via `ASSISTIU`; Intercontinental (filme) está relacionado a 2003-08-22 (filme) via `ASSISTIU`; Intercontinental (filme) está relacionado a 2003-05-10 (filme) via `ASSISTIU`.

Started streaming 4 records after 1 ms and completed after 1 ms.



- **Usuario:** id, nome, email, data_nascimento
- **Filme:** id, nome, ano
- **Genero:** id, nome

A recomendação é gerada assim: encontra o filme com a **nota mais alta** dada pelo usuário, pega o(s) gênero(s) desse filme, e sugere outros filmes do(s) mesmo(s) gênero(s) que o usuário ainda não assistiu.

Como rodar localmente:

Pré-requisitos

- Python 3.12+
- Docker Desktop
- Uma instância Neo4j Aura (free tier) já criada

1. Clonar o repositório

No PowerShell:

```
git clone
https://github.com/Danillo-Vidal/Plataforma-de-Recomenda-o-com-Neo4j-Redis-e-Python-.
git
cd Plataforma-de-Recomenda-o-com-Neo4j-Redis-e-Python-
code .
```

2. Inicializar container

Confirme que o `docker-compose.yml` está na raiz do projeto clonado e rode:

```
docker compose up -d
```

3. Configurar o `.env`

Atualize o arquivo `.env` na raiz do projeto com as credenciais da sua instância Neo4j Aura:

```
NEO4J_URI=neo4j+s://xxxxxxx.databases.neo4j.io
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=sua-senha-aqui
NEO4J_DATABASE=neo4j

REDIS_HOST=localhost
REDIS_PORT=6379
```

Caso não tenha essas credenciais salvas em outro lugar, pegue de novo no console do Aura.

4. Criar o ambiente virtual e instalar as dependências

```
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
```

5. Testar a conexão

```
python -c "from dotenv import load_dotenv; import os; load_dotenv();
print(repr(os.getenv('NEO4J_URI')))"
```

Se retornar a URI certa (não None), sobe o servidor:

```
uvicorn app.main:app --reload
```

Abra `http://127.0.0.1:8000/docs` no navegador para ver e testar os endpoints (Swagger).

Endpoints disponíveis (API)

Método	Rota	O que faz
GET	/	Health check
GET	/test/neo4j	Testa conexão com Neo4j
GET	/test/redis	Testa conexão com Redis
POST	/usuarios	Cadastra usuário
PUT	/usuarios/{usuario_id}	Atualiza usuário
POST	/filmes	Cadastra filme
PUT	/filmes/{filme_id}	Atualiza filme
DELETE	/filmes/{filme_id}	Remove filme (e suas relações)
POST	/generos	Cadastra gênero
POST	/filmes/{filme_id}/generos/{genero_id}	Associa um gênero a um filme
POST	/assistidos	Registra filme assistido
POST	/avaliacoes	Registra avaliação (nota 0-10)
GET	/recomendacoes/{usuario_id}	Gera/retorna recomendações (com cache)
DELETE	/recomendacoes/{usuario_id}/cache	Invalida o cache de recomendação

Popular o banco (script Cypher)

Cole no Query do Neo4j Aura para criar os dados de teste:

```
CREATE (:Usuario {id: 1, nome: 'Dan', email: 'dan@teste.com', idade: 22})
CREATE (:Usuario {id: 2, nome: 'Mari', email: 'mari@teste.com', idade: 22})
CREATE (:Usuario {id: 3, nome: 'Matheus', email: 'teteuze@teste.com', idade: 18})

CREATE (:Filme {id: 1, nome: 'Matrix'})
CREATE (:Filme {id: 2, nome: 'Interestelar'})
CREATE (:Filme {id: 3, nome: 'Duna'})

CREATE (:Genero {nome: 'Ficção Científica'})
CREATE (:Genero {nome: 'Drama'})
CREATE (:Genero {nome: 'Ação'})

MATCH (g:Genero {nome:'Ficção Científica'}) SET g.id = 1;
MATCH (g:Genero {nome:'Drama'}) SET g.id = 2;
MATCH (g:Genero {nome:'Ação'}) SET g.id = 3;
```

```

MATCH (f:Filme {id: 2}) MATCH (g:Genero {nome: 'Drama'}) CREATE
(f)-[:PERTENCE_A]->(g);
MATCH (f:Filme {id: 3}) MATCH (g:Genero {id: 1}) CREATE (f)-[:PERTENCE_A]->(g);
MATCH (f:Filme {id: 1}) MATCH (g:Genero {id: 3}) CREATE (f)-[:PERTENCE_A]->(g);

MATCH (u:Usuario{id: 1}) MATCH (f:Filme {id: 3}) CREATE (u)-[:AVALIOU {nota:
10}]->(f);
MATCH (u:Usuario{id: 2}) MATCH (f:Filme {id: 1}) CREATE (u)-[:AVALIOU {nota:
10}]->(f);
MATCH (u:Usuario{id: 2}) MATCH (f:Filme {id: 2}) CREATE (u)-[:AVALIOU {nota:
5}]->(f);

MATCH (u:Usuario{id: 2}) MATCH (f:Filme {id: 2}) CREATE (u)-[:ASSISTIU]->(f);
Cria 3 usuários, 3 filmes, 3 gêneros e algumas relações de exemplo, suficiente para testar GET
/recomendacoes/1.

```

Estratégia de cache (Redis)

- **Chave:** recomendacao:{usuario_id}
- **TTL:** 300 segundos
- **HIT:** chave existe no Redis → retorna direto, sem consultar o Neo4j
- **MISS:** chave não existe (nunca calculada, expirou ou foi invalidada) → consulta o Neo4j, salva no Redis e retorna
- **Invalidação manual:** DELETE /recomendacoes/{usuario_id}/cache remove a chave antes do TTL expirar

Testes de desempenho

Script testes_performance.py: mede o tempo de GET /recomendacoes/{usuario_id} em dois cenários.

Com a API rodando, em outro terminal:

```
python -m app.testes_performance
```

O script:

1. Invalida o cache do usuário de teste, garantindo um MISS limpo
2. Faz 1 chamada e mede o tempo (MISS — consulta o Neo4j Aura)
3. Faz 20 chamadas seguidas e mede o tempo de cada uma (HIT — servidas só pelo Redis)
4. Imprime tempo médio, mínimo e máximo de cada grupo

Como interpretar: o MISS reflete latência de rede até o Neo4j Aura + execução da query + escrita no Redis. O HIT reflete só a leitura no Redis local (1-5ms esperado). Ganho esperado: uma ordem de grandeza ou mais no tempo de resposta, e redução do número de consultas ao Neo4j — o objetivo pedido pelo enunciado.

```
PS C:\Users\maric\OneDrive\Documentos\Projeto do DAN\Plataforma-de-Recomenda-o-com-Neo4j-Redis-e-Python\app> python testes_performance.py
1ª chamada (deve ser CACHE MISS):
status: 200 | tempo: 1953.70 ms | corpo: {"usuario_id":1,"origem":"CACHE MISS","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
MISS: média 1953.70 ms | min 1953.70 ms | max 1953.70 ms

Próximas 20 chamadas (devem ser CACHE HIT):
status: 200 | tempo: 5.20 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 8.76 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 10.90 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 21.62 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 67.17 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 15.37 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 10.01 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 12.22 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 17.19 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 16.80 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 12.42 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 8.75 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 6.17 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 7.17 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 7.48 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 11.34 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 14.97 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 12.45 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 13.16 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
status: 200 | tempo: 8.47 ms | corpo: {"usuario_id":1,"origem":"CACHE HIT","recomendacoes":[{"titulo":"Duna","genero":"Ficção Científica"}]}
HIT: média 14.38 ms | min 5.20 ms | max 67.17 ms
PS C:\Users\maric\OneDrive\Documentos\Projeto do DAN\Plataforma-de-Recomenda-o-com-Neo4j-Redis-e-Python\app> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
```

**rodando scirpt do Redis*

Cenário	Tempo médio	Mín	Máx
1ª chamada (CACHE MISS)	1953.70 ms	1953.70 ms	1953.70 ms
20 chamadas seguintes (CACHE HIT)	14.38 ms	5.20 ms	67.17 ms

Valores medidos em execução real (Windows/PowerShell, Neo4j Aura + Redis local): a 1ª chamada (MISS) levou 1953.70 ms, contra uma média de 14.38 ms nas 20 chamadas seguintes (HIT) — uma redução de mais de 100x no tempo de resposta e zero consultas adicionais ao Neo4j enquanto o cache está válido.